HNC-Benchmark TODO 开发方案

供 VSCode + Claude Code 使用

版本：v1.0 | 2026-04-01

读取顺序：先读 hnc-benchmark-context-v4.md，再读本文件

首要任务：数据集处理（当前阶段）

在做任何网站、服务器、测评系统之前，首先要有干净可用的数据集。 当前最紧迫的工作是：**从 Hancock 原始数据生成我们自己的评测数据集，再交给专家标注/检查。**

PHASE 0：数据处理（当前首要，约2-3周）

P0-1 环境搭建与数据下载

```
[ ] 申请 TCIA 账号，获取 Hancock 数据集访问权限
    URL: https://www.cancerimagingarchive.net/collection/hancock/
    需要 IBM Aspera Connect 插件下载（4.5TB总量）
    按需下载策略：
      - 优先下载：StructuredData（JSON，约几MB）
      - 其次下载：TextData/reports_english + histories_english（约几百MB）
      - 按患者ID批量下载 WSI（按需，约先下载50例用于开发调试）
      - GeoJSON标注文件（约几十MB）

[ ] 本地开发环境
    Python >= 3.10
    pip install openslide-python Pillow numpy pandas anthropic tqdm
    安装 openslide 系统库（Linux: apt-get install openslide-tools）
```

P0-2 结构化数据解析与字段映射

```
[ ] 解析 clinical_data.json + pathological_data.json
    输出：每个患者的字段完整性报告
    重点检查字段：pT_stage, pN_stage, resection_status, perinodal_invasion,
                  lymphovascular_invasion_L, vascular_invasion_V,
                  perineural_invasion_Pn, grading_hpv, infiltration_depth_in_mm,
                  closest_resection_margin_in_cm, histologic_type,
                  adjuvant_radiotherapy, adjuvant_systemic_therapy,
```

```
                    adjuvant_chemoradiotherapy, adjuvant_systemic_therapy_mode

[ ] 声门喉癌 vs 声门上喉癌区分脚本
    输入: icd_codes 字段
    ICD规则: C32.0=声门喉癌(Glottic); C32.1=声门上喉癌(Supraglottic); C32.2=声门下;
C32.9=喉癌NOS
    输出: 新增字段 cancer_subtype (写入 meta.json)
    注意: C32.9(NOS)需要专家人工确认归类

[ ] 数据矛盾检测脚本
    检测逻辑: adjuvant_systemic_therapy=no 但 adjuvant_systemic_therapy_mode 有值
              adjuvant 字段全为 no 但 adjuvant_treatment_intent=curative
    输出: 矛盾案例列表 (供专家修正)

[ ] 字段完整性统计报告
    统计每个字段在763例中的缺失率
    输出: completeness_report.csv (字段名, 总数, 缺失数, 缺失率, null值示例)
```

P0-3 NCCN 路径节点自动标注

```
[ ] 编写 NCCN 路径规则引擎 (nccn_router.py)
    输入: primary_tumor_site, cT_stage, cN_stage, p16_status, M_status
    输出: nccn_pathway_node (如 HYPO-3)

    规则表 (严格按照 context-v4.md 第6节, 不得自行修改):
    - 口腔癌: T1-2N0→OR-2; T3N0/T1-3N1-3/T4aN0-3→OR-3; T4b/无法切除→ADV-1; M1→ADV-2
    - 口咽癌p16-: T1-2N0-1→ORPH-2; T3-4aN0-1→ORPH-3; T1-4aN2-3→ORPH-4; T4b→ADV-1;
M1→ADV-2
    - 口咽癌p16+: M1→ADV-2; T1-2N0→ORPHPV-1; T0-2N1(≤3cm)→ORPHPV-2;
                  T0-2N1(>3cm或≥2个≤6cm)/T1-2N2/T3N0-2→ORPHPV-3; T0-3N3/T4N0-
3→ORPHPV-4
    - 下咽癌: 适合喉保留(多T1N0,部分T2N0)→HYPO-2; T1-3N0-3→HYPO-3;
              T4aN0-3→HYPO-5; T4b/无法切除/不适合手术→ADV-1; M1→ADV-2
    - 声门喉癌: 原位癌/适合保留(T1-2N0,选定T3N0)→GLOT-2;
                T3全喉N0-1→GLOT-3; T3全喉N2-3→GLOT-4; T4a→GLOT-6; T4b→ADV-1;
M1→ADV-2
    - 声门上喉癌: 适合保留(多T1-2N0,部分T3)→SUPRA-2; T3N0全喉→SUPRA-3;
                  T4aN0→SUPRA-8; N+→SUPRA-4分流(T1-2N+/部分T3N1→SUPRA-5;
                  多数T3N1-3→SUPRA-6; T4aN1-3→SUPRA-8); T4b→ADV-1; M1→ADV-2

[ ] 辅助治疗决策规则引擎 (adjuvant_router.py)
    输入: nccn_pathway_node, pT_stage, pN_stage, perinodal_invasion,
          resection_status, closest_resection_margin_in_cm,
          lymphovascular_invasion_L, perineural_invasion_Pn
    输出:
      guideline_adjuvant_RT: yes/no/consider
      guideline_adjuvant_chemoRT: yes/no (1类触发条件: ENE or R1/margin<0.1cm)
      adjuvant_decision_trigger: [列出触发的特征]
      recommendation_category: 1类/2A类/2B类/无
    规则严格按照 context-v4.md 第7节各类型决策流
```

P0-4 describe_focus 生成

```
[ ] 编写 describe_focus 生成脚本（generate_describe_focus.py）
    方法：调用 Claude API，按 tumor_site + cT + cN + histologic_type 生成影像学描述

    Prompt 模板（示例）：
    "你是放射科专家。根据以下病理信息，生成一段标准化的影像学描述（CT或超声），
    描述应只包含支持该分期的影像学征象，不得透露pT/pN字样（只用影像描述）。
    肿瘤部位：{tumor_site}，临床T分期：{cT}，临床N分期：{cN}，组织类型：
{histologic_type}。
    100-150字，中文，仅输出描述文本。"

    输出：describe_focus字段写入 step1_input.json
    标记字段：describe_focus_source: "ai_generated_pending_review"

    注意：生成后不自动使用，必须经专家检查后将标记改为 "expert_reviewed"

[ ] 泄露信息检测脚本（detect_cTcN_leak.py）
    扫描手术报告和病史文本，检测是否含有 cT/cN 明确描述
    关键词：cT1，cT2，cT3，cT4，cN0，cN1，cN2，cN3，临床分期，clinical staging
    输出：含泄露的文件列表 + 泄露片段（供人工删除）
    将结果写入 meta.json: contains_cTcN_leak: true/false

[ ] 辅助治疗建议删除脚本（clean_surgery_reports.py）
    检测手术报告中医生直接给出辅助治疗建议的段落
    常见模式：建议术后放疗、推荐辅助化疗、adjuvant...
    输出：标记需人工核实删除的段落（不自动删除，需专家确认）
```

P0-5 WSI Patch 提取

```
[ ] 编写 WSI patch 提取脚本（extract_wsi_patches.py）
    依赖：openslide-python，Pillow，numpy

    输入：
    - WSI 文件路径（.svs）
    - GeoJSON 标注文件路径

    输出（每个患者）：
    - primary_5x_scalebar.jpg     （全景，0.5μm/px等效5x，加比例尺100μm）
    - primary_20x_scalebar.jpg    （细节，0.5μm/px等效20x，加比例尺50μm）
    - lymphnode_10x_scalebar.jpg  （淋巴结，1.0μm/px等效10x，加比例尺100μm，若无则记录
原因）

    Tier1（有GeoJSON标注时）：
    - 读取GeoJSON多边形坐标
    - 计算外接矩形 + 10%边距
    - 用 openslide.read_region() 在目标分辨率提取

    Tier2（无标注时fallback）：
    - 生成缩略图，检测组织区域（非白色背景）
```

```
    - 从组织高密度区采样6个tile

    比例尺绘制:
    - 计算 100μm 对应的像素数 = 100 / target_mpp
    - 在图像右下角绘制实心白色矩形+黑色文字标注

[ ] 淋巴结WSI缺失记录脚本 (record_ln_wsi_status.py)
    检查每个患者的 pN_stage
    pN0 → ln_wsi_missing_reason: "pN0"
    pN+ 但文件不存在 → ln_wsi_missing_reason: "specimen_unavailable"
    写入 meta.json
```

P0-6 完整 Case JSON 生成

```
[ ] 编写 case_builder.py
    整合以上所有步骤，为每个患者生成完整的 Case 目录结构

    step1_input.json 字段:
      year_of_initial_diagnosis, age_at_initial_diagnosis, sex, smoking_status,
      primarily_metastasis, primary_tumor_site, icd_code,
      describe_focus (AI生成待审核) , medical_history
      【不包含】hpv_association_p16, adjuvant_treatment_intent

    step1_gt.json:
      current_phase: "CLINICAL_STAGING"
      cT_stage (来自pathological_data.json的cT字段)
      cT_evidence_anchor (AI提取待审核)
      cN_stage, cN_evidence_anchor
      p16_subask_required: true/false (口咽癌=true)

    step2_gt.json:
      nccn_pathway_node (规则引擎自动生成)
      recommendation_category (按节点填写，无等级的节点不填)
      recommended_treatment_options (按节点固定文本)
      surgery_detail (按节点固定文本)

    step3_input.json:
      surgery_report (已清洗版本) , medical_history,
      icd_code, ops_codes,
      number_of_positive_lymph_nodes, number_of_resected_lymph_nodes

    step3_gt.json:
      current_phase: "SURGERY_GROSS_PATHOLOGY"
      preliminary_pT_stage, pT_anchor (AI提取待审核)
      preliminary_resection_status, resection_anchor (AI提取待审核)
      preliminary_histologic_type, histotype_anchor (AI提取待审核)

    step4_gt.json (直接来自 pathological_data.json) :
      updated_pT_stage, updated_pN_stage, updated_resection_status,
      updated_histologic_type, perineural_invasion_Pn,
      lymphovascular_invasion_L, vascular_invasion_V, grading_hpv,
```

```
        carcinoma_in_situ, resection_status_carcinoma_in_situ,
        infiltration_depth_in_mm, closest_resection_margin_in_cm,
        perinodal_invasion, staging_upgraded (cT vs pT自动比较)

      step5_gt.json:
        adjuvant_treatment_intent (来自数据集, 矛盾已标记)
        adjuvant_radiotherapy_y_n, adjuvant_radiotherapy_type,
        adjuvant_systemic_therapy_y_n, adjuvant_systemic_therapy_mode,
        adjuvant_chemoradiotherapy_y_n
        guideline_adjuvant_RT, guideline_adjuvant_chemoRT (规则引擎生成)
        patient_declined (若数据与指南偏差则标记, 待专家确认)

  [ ] 生成专家标注工作表 (expert_review_tasks.xlsx 或 .csv)
      包含以下待审核项:
      - describe_focus 审核列表 (AI生成, 需确认或修改)
      - evidence anchor 审核列表 (AI提取, 需确认)
      - 泄露信息删除列表 (需手动删除)
      - 矛盾数据修正列表 (类型A/B)
      - ICD codes 声门/声门上归类确认列表 (尤其C32.9 NOS)
      - NCCN路径节点抽样验证列表 (20%抽样)
      - 指南偏差案例列表 (adjuvant与指南推荐不符)
```

P0-7 数据集质量验证

```
  [ ] 统计最终数据集完整性
      目标: 生成 dataset_stats_report.md
      内容:
      - 各癌种患者数量
      - 每步GT字段的完整率
      - WSI patch 提取成功率
      - 各字段null值数量
      - 待专家审核项总数

  [ ] 案例难度分级脚本
      Simple: pT1-2, pN0, 无不良特征
      Medium: pT3, pN1, 1个不良特征
      Hard: pT4, pN2-3, 多个不良特征 or 升期
      输出: difficulty字段写入 meta.json

  [ ] 生成 450 例核心评测集
      按癌种、难度分层抽样
      输出: benchmark_cases_450.json (case_id列表)
```

PHASE 1: 评测引擎 (数据集完成后)

P1-1 评分器实现

```
[ ] structured_scorer.py
    实现所有结构化字段的自动评分（按 context-v4.md 第8节规则）
    支持：精确匹配、相邻分期、数值范围、语义范围等

[ ] llm_judge.py
    封装 Judge LLM 调用
    输入：字段名、字段值、评分标准列表、原始输入上下文
    输出：每条标准的 T/F 结果 + 总分
    调用 claude-sonnet-4-6 API

[ ] safety_checker.py
    检测高危错误：PERINODAL_MISS, CHEMORT_MISSED, TEMPORAL_VIOLATION,
                   HALLUCINATION_FLAG, LN_WSI_HALLUC, CASCADE_ERROR
    输出：CSS 分数 + flag 列表

[ ] case_evaluator.py
    整合三个评分器
    支持 AR mode（级联传播）和 IND mode（每步注GT）
    输出：完整评分报告 JSON
```

P1-2 被测模型接口

```
[ ] model_runner.py
    统一接口，支持调用各种模型 API：
    - OpenAI（GPT-4o, GPT-4V）
    - Anthropic（Claude claude-sonnet-4-6）
    - Google（Gemini Pro Vision）
    - 本地部署模型（LLaMA, Qwen-VL 等，通过 ollama 或 vllm）
    处理 WSI 图像输入（base64 编码，大小限制处理）

[ ] wsi_image_encoder.py
    将 WSI patches 编码为各 API 要求的格式
    处理大图片分割（若超过 API 限制）
    注：Claude API 单图上限 5MB，Prompt 内最多 20 张图
```

P1-3 完整评测流程

```
[ ] run_benchmark.py
    主入口：输入模型名称 + 评测集路径
    输出：完整评测结果 results/{model_name}/
    支持断点续跑（已评测的 case 不重复）
    进度显示（tqdm）

[ ] results_aggregator.py
    汇总所有 case 的分数
    输出：leaderboard_entry.json（供网站展示）
```

PHASE 2：基础设施（与PHASE 1并行）

P2-1 服务器选型与部署

```
架构方案：
  数据存储服务器（存 Hancock WSI + benchmark 数据集）：
    推荐：阿里云/腾讯云对象存储（OSS/COS）+ 1台轻量应用服务器
    存储量：benchmark 数据集约 50GB（450例 × patches）+ 原始数据按需

  评测计算服务器（运行评测流程）：
    推荐：按需租用 GPU 服务器（评测不需要持续运行，按次付费）
    或：本地运行，结果上传到存储服务器

  网站服务器：
    推荐：1核2GB 轻量服务器即可（只展示结果）

[ ] 对象存储配置
    bucket 结构：
    hnc-benchmark/
    ├── dataset/cases/{case_id}/    # 评测数据集（不含原始WSI）
    ├── results/{model_name}/       # 评测结果
    └── public/leaderboard.json     # 公开排行榜数据

[ ] 数据访问控制
    评测数据集：需注册申请（参考 Hancock 的访问控制方式）
    排行榜数据：公开
    原始WSI：本地存储，不上传（4.5TB）
```

P2-2 公开网站

```
技术栈推荐：
  前端：Next.js + Tailwind CSS（支持静态导出，可本地部署）
  后端：FastAPI（Python，与评测代码同语言）
  数据库：SQLite（轻量，排行榜和结果存储足够）

[ ] 网站页面设计
    /                    # 项目主页（介绍、方法论）
    /leaderboard         # 排行榜（模型名、总分、各步骤分、CSS分）
    /dataset             # 数据集介绍（统计信息、访问申请）
    /paper               # 论文/技术报告链接
    /docs                # 评测方法说明
    /submit              # 提交模型评测请求

[ ] 本地部署支持
    docker-compose.yml（一键启动网站+数据库）
    README 包含本地部署步骤
    不依赖外部服务（数据库用SQLite，不需要云服务也能跑）

[ ] 排行榜设计
    列：模型名 | 总分 | Step1 | Step2 | Step3 | Step4 | Step5 | CSS | AR/IND | 测评
```

```
日期
    支持按列排序、按模型类型筛选（专业医疗模型 vs 通用模型）
    每个模型可展开查看分步详情
```

PHASE 3：持续迭代（后续）

```
[ ] 增加更多评测案例（从剩余763例中扩充）
[ ] 支持更多模型
[ ] 专家反馈收集机制
[ ] 定期更新排行榜
[ ] 论文写作（方法论 + 实验结果）
```

当前首要工作清单（立即开始）

按优先级排序，首要目标是生成可供专家审核的数据集：

```
优先级1（本周）：
  [x] 下载 StructuredData（几MB，立即可得）
  [ ] P0-2：字段映射与完整性统计
  [ ] P0-2：声门/声门上 ICD 区分脚本
  [ ] P0-2：数据矛盾检测脚本

优先级2（第2周）：
  [ ] P0-3：NCCN路径规则引擎（nccn_router.py）
  [ ] P0-3：辅助治疗决策规则引擎（adjuvant_router.py）
  [ ] P0-4：describe_focus AI生成脚本

优先级3（第2-3周，需要 WSI 文件）：
  [ ] P0-5：WSI patch 提取脚本（先用50例测试）
  [ ] P0-6：完整 Case JSON 生成器（case_builder.py）
  [ ] P0-6：生成专家标注工作表

优先级4（第3周，数据集基本完成后）：
  [ ] P0-7：数据集质量验证报告
  [ ] P0-7：450例核心评测集划分
  [ ] 将数据集交给专家审核
```

项目目录结构（建议）

```
hnc-benchmark/
├── README.md
├── hnc-benchmark-context-v4.md       # 本项目上下文（Claude读取用）
├── hnc-benchmark-todo.md             # 本TODO文件
```

```
│
├── data_processing/                  # PHASE 0
│   ├── parse_hancock.py              # P0-2: 解析结构化数据
│   ├── classify_larynx_icd.py        # P0-2: 声门/声门上区分
│   ├── detect_data_issues.py         # P0-2: 矛盾检测
│   ├── nccn_router.py                # P0-3: NCCN路径规则引擎
│   ├── adjuvant_router.py            # P0-3: 辅助治疗决策引擎
│   ├── generate_describe_focus.py    # P0-4: AI生成影像描述
│   ├── detect_cTcN_leak.py           # P0-4: 泄露检测
│   ├── clean_surgery_reports.py      # P0-4: 报告清洗
│   ├── extract_wsi_patches.py        # P0-5: WSI提取
│   ├── case_builder.py               # P0-6: Case生成器
│   └── validate_dataset.py           # P0-7: 质量验证
│
├── evaluation/                       # PHASE 1
│   ├── structured_scorer.py
│   ├── llm_judge.py
│   ├── safety_checker.py
│   ├── case_evaluator.py
│   ├── model_runner.py
│   └── run_benchmark.py
│
├── website/                          # PHASE 2
│   ├── frontend/                     # Next.js
│   ├── backend/                      # FastAPI
│   ├── docker-compose.yml
│   └── README_deploy.md
│
├── dataset/                          # 生成的评测数据集
│   ├── cases/
│   │   ├── Case_0001/
│   │   └── ...
│   ├── benchmark_cases_450.json
│   └── dataset_stats_report.md
│
└── expert_review/                    # 待专家审核的文件
    ├── describe_focus_review.csv
    ├── evidence_anchor_review.csv
    ├── data_contradiction_list.csv
    ├── icd_larynx_review.csv
    └── adjuvant_deviation_list.csv
```

TODO文件结束 | 开始时告诉 Claude：当前在执行哪个 P0-X 任务